

Longitudinal Data Analysis

Efficient Bayesian Analysis of Longitudinal and Survival Data Using R-INLA
ISCB 2026, Freiburg, Germany

Denis Rustand

September 27, 2026

```
library(INLA)
library(INLAjoint)
library(ggplot2)
library(ggpubr)
```

Longitudinal data

```
set.seed(1)
n <- 30
Sigma <- matrix(c(0.8, 0.2, 0.2, 0.4), 2) # true RE var-cov: int .8, slope .4, cov .2
b <- MASS::mvrnorm(n, c(0, 0), Sigma)
simDat <- do.call(rbind, lapply(seq_len(n), function(i) {
  t <- sort(runif(4, 0, 2)) # few visits, short follow-up
  data.frame(id = i, time = t,
             Y = 3 + b[i, 1] + (-0.5 + b[i, 2]) * t + rnorm(4, 0, 0.6))
}))
head(simDat)
```

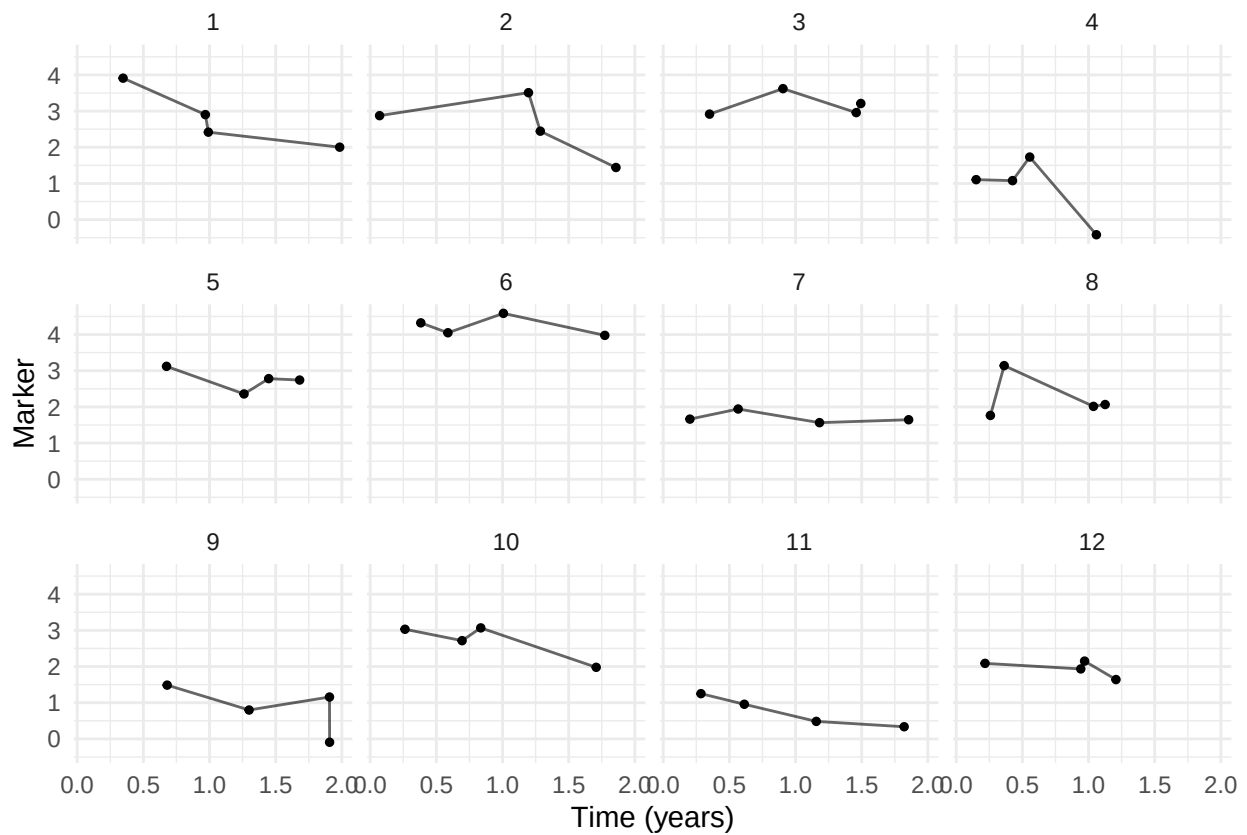
```
##   id      time      Y
## 1  1 0.34688467 3.909943
## 2  1 0.96869905 2.902486
## 3  1 0.99118716 2.417645
## 4  1 1.98367724 2.002591
## 5  2 0.07108116 2.873951
## 6  2 1.19618484 3.508892
```

Each row is one visit, not one patient.

A few patients

```
ids_show <- unique(simDat$id)[1:12]

ggplot(simDat[simDat$id %in% ids_show, ], aes(x = time, y = Y)) +
  facet_wrap(~id) +
  geom_point(size = 1) +
  geom_line(alpha = 0.6) +
  labs(x = "Time (years)", y = "Marker") +
  theme_minimal()
```



Linear regression

As a first pass, we can fit an ordinary linear model.

```
M_lm <- lm(Y ~ time, data = simDat)
summary(M_lm)
```

```
##
## Call:
## lm(formula = Y ~ time, data = simDat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.6974 -0.7678 -0.0965  0.5852  3.4657
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.0381     0.2316  13.119 < 2e-16 ***
## time          -0.7164     0.2056  -3.485 0.000692 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.273 on 118 degrees of freedom
## Multiple R-squared:  0.09331,    Adjusted R-squared:  0.08562
```

```
## F-statistic: 12.14 on 1 and 118 DF, p-value: 0.0006922
```

Same with INLAjoint:

```
M_lm2 <- joint(formLong = Y ~ time,
               dataLong = simDat, id = "id", timeVar = "time")
summary(M_lm2) # sdcor=T for std dev instead of variance
```

```
## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept      3.0362 0.2311      2.5822  3.0362    3.4901
## time           -0.7148 0.2052     -1.1178 -0.7149   -0.3118
## Res. err. (variance) 1.6212 0.2099      1.2600  1.6039    2.0843
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -217.7912                      -218.1768
##
## Deviance Information Criterion: 402.5336
## Widely applicable Bayesian information criterion: 402.8046
## Computation time: 0.9 seconds
```

Linear mixed-effects models

Random intercept model

First, allow each patient to have their own baseline level:

$$\eta_i(t) = (\beta_0 + b_{i0}) + \beta_1 t.$$

```
M_ri <- joint(
  formLong = Y ~ time + (1 | id),
  dataLong = simDat,
  id = "id",
  timeVar = "time"
)
summary(M_ri, sdcor = TRUE)
```

```
## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept      2.9518 0.2362      2.4865  2.9520    3.4160
## time           -0.6292 0.1131     -0.8516 -0.6291   -0.4072
## Res. err. (sd)  0.6313 0.0465      0.5463  0.6290    0.7290
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept 1.0942 0.1508      0.8329  1.082    1.425
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -182.6548                      -182.2198
##
## Deviance Information Criterion: 261.5798
## Widely applicable Bayesian information criterion: 263.3044
## Computation time: 0.62 seconds
```

The random intercept standard deviation tells us how much patients differ at baseline after accounting for time.

```
head(M_ri$summary.random$IDIntercept_L1)
```

##	ID	mean	sd	0.025quant	0.5quant	0.975quant	mode	kld
## 1	1	0.4894043	0.3549144	-0.2062151	0.4894043	1.1850237	0.4894043	0
## 2	2	0.2845479	0.3550166	-0.4112719	0.2845479	0.9803676	0.2845479	0
## 3	3	0.8165466	0.3548593	0.1210353	0.8165466	1.5120580	0.8165466	0
## 4	4	-1.5975664	0.3574891	-2.2982323	-1.5975664	-0.8969006	-1.5975664	0
## 5	5	0.5486110	0.3560503	-0.1492349	0.5486110	1.2464568	0.5486110	0
## 6	6	1.7248694	0.3547879	1.0294979	1.7248694	2.4202409	1.7248694	0

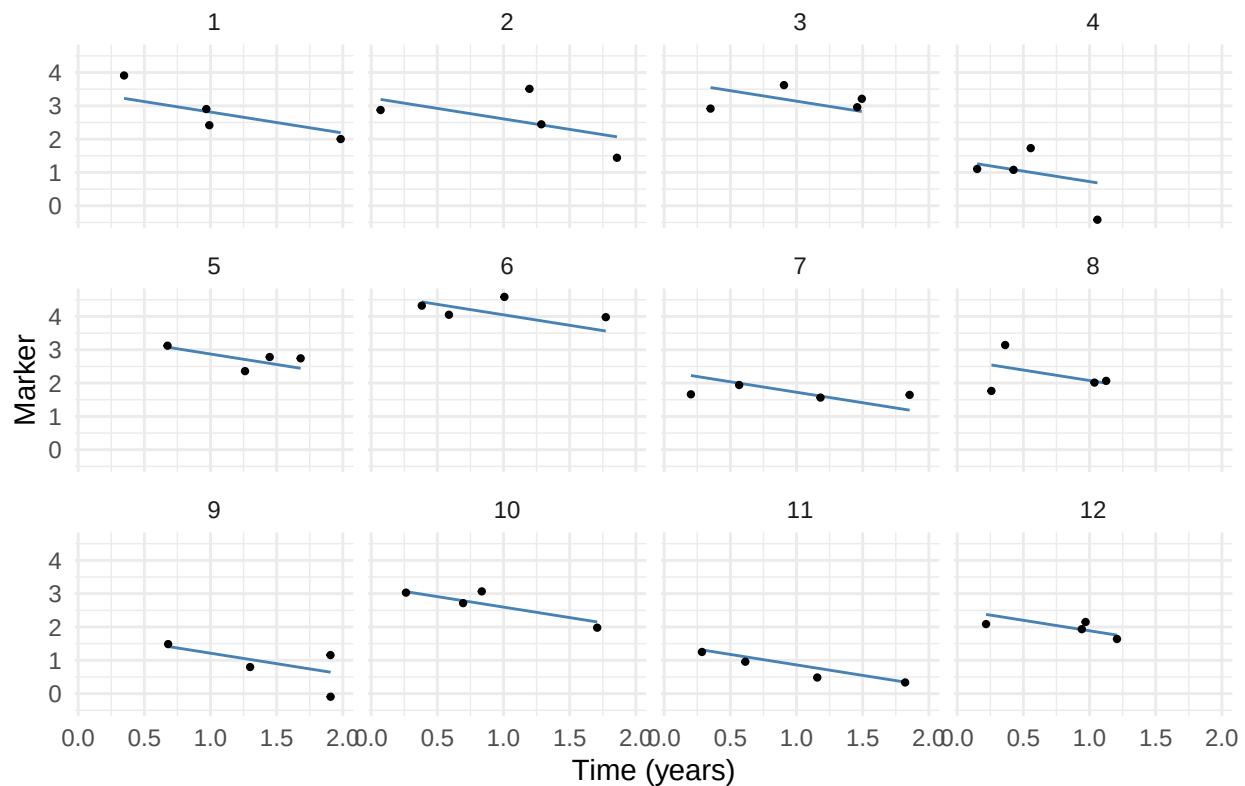
Individual fits from the random intercept model

`summary.fitted.values` gives the fitted values at the observed visits, including the random effects. Here the rows follow the order of the data used to fit the model.

```
fit_ri <- cbind(simDat, M_ri$summary.fitted.values)
```

```
ggplot(fit_ri[fit_ri$id %in% ids_show, ], aes(x = time, y = mean)) +  
  facet_wrap(~id) +  
  geom_line(color = "steelblue") +  
  geom_point(data = simDat[simDat$id %in% ids_show, ],  
             aes(x = time, y = Y), size = 0.8) +  
  labs(x = "Time (years)", y = "Marker",  
       title = "Random intercept model") +  
  theme_minimal()
```

Random intercept model



The fitted lines are shifted up or down by patient, but their slopes remain parallel.

Is it realistic that every patient changes at the same rate?

Random intercept and slope model

Now each patient can also have their own rate of change:

$$\eta_i(t) = (\beta_0 + b_{i0}) + (\beta_1 + b_{i1})t.$$

```
M_rs <- joint(
  formLong = Y ~ time + (1 + time | id),
  dataLong = simDat,
  id = "id",
  timeVar = "time"
)
summary(M_rs, sdcor = TRUE)
```

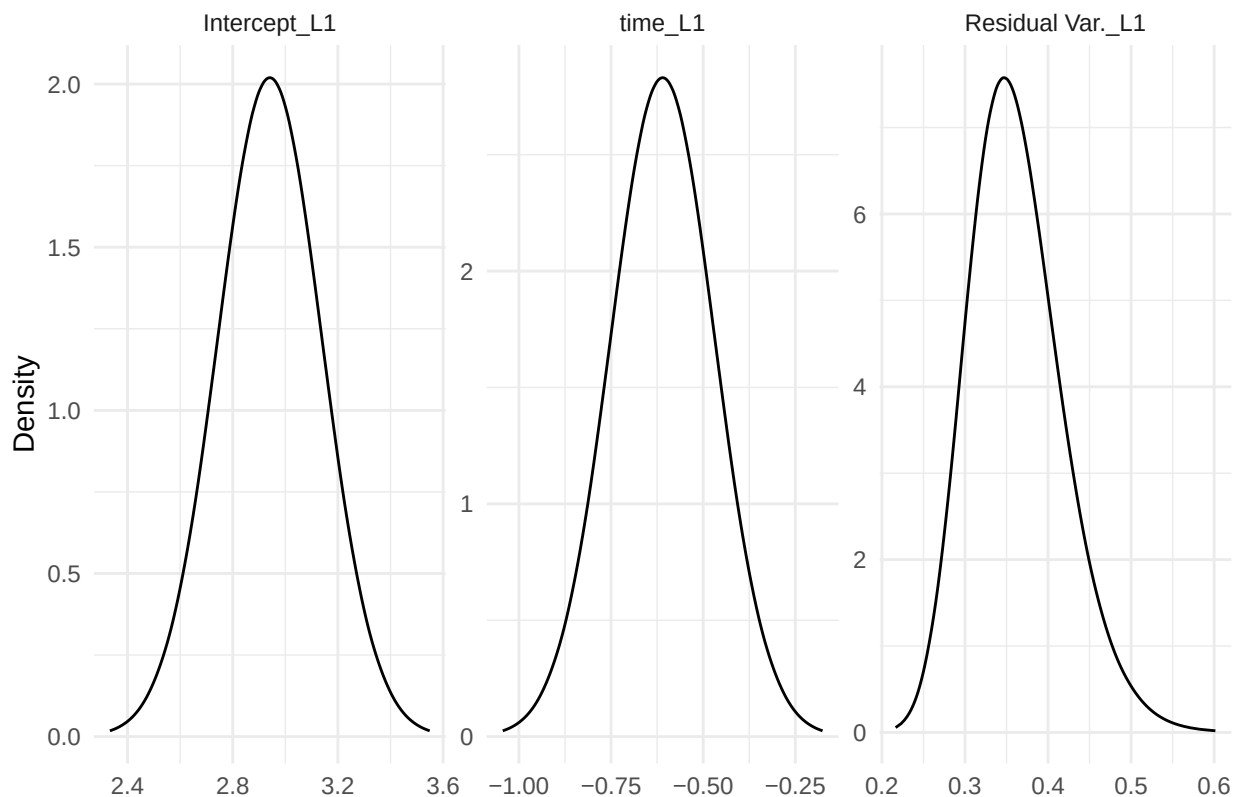
```
## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept      2.9412 0.2065    2.5335    2.9413    3.3480
## time          -0.6106 0.1442   -0.8947   -0.6105   -0.3268
## Res. err. (sd)  0.5998 0.0461    0.5153    0.5977    0.6966
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept      0.9101 0.1778    0.6250    0.8897    1.3138
```

```
## time          0.4925 0.1075    0.3113  0.4804    0.7338
## Intercept:time 0.3611 0.2637   -0.2072  0.3932    0.7740
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                                     -176.0535                                -173.8339
##
## Deviance Information Criterion: 258.4789
## Widely applicable Bayesian information criterion: 257.7374
## Computation time: 0.6 seconds
```

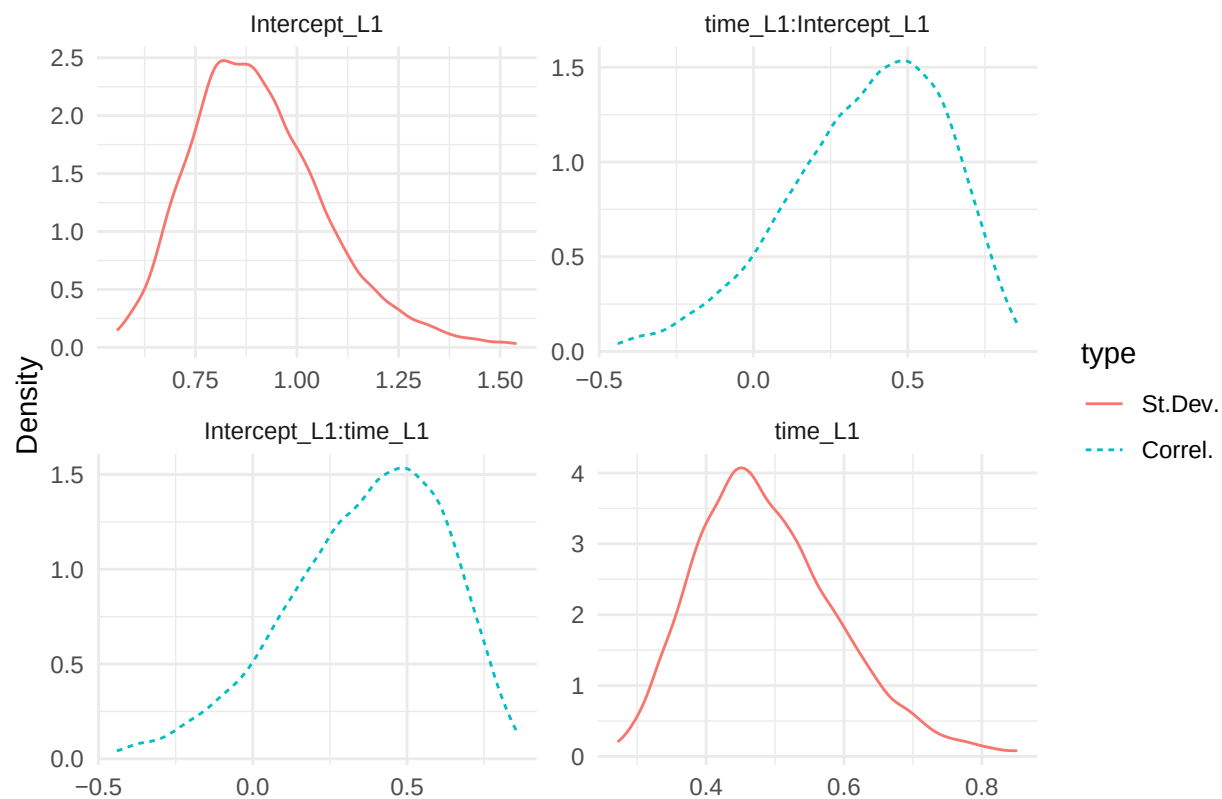
Now the model estimates a random intercept, a random slope and their correlation. The `sdcor = TRUE` argument is useful: standard deviations and correlations are much easier to interpret than variances and covariances.

The same information can be inspected graphically. `plot()` returns a list of `ggplot2` objects, so we can print the component we want and still modify it with ordinary `ggplot2` syntax if needed.

```
plot(M_rs)$Outcomes$L1
```



```
plot(M_rs, sdcor = TRUE)$Covariances$L1
```

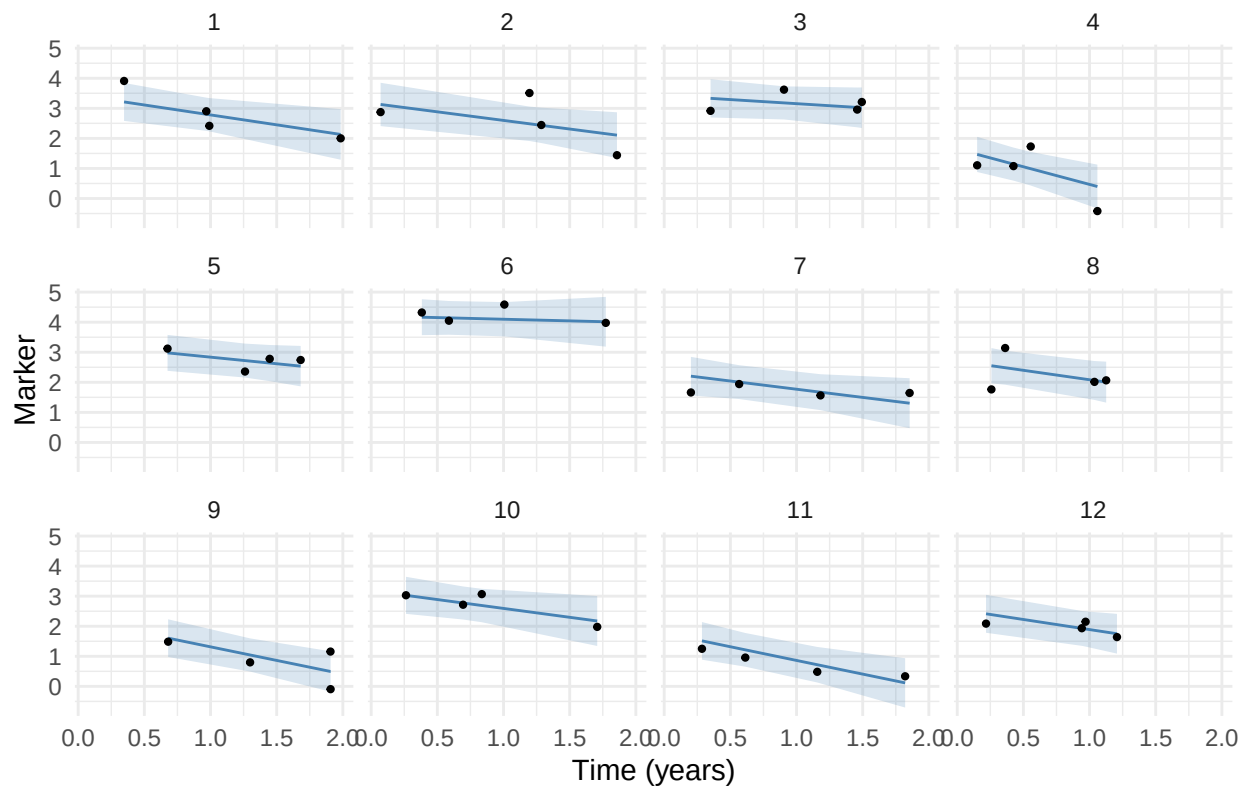


Individual fits from the random slope model

```
fit_rs <- cbind(simDat, M_rs$summary.fitted.values)

ggplot(fit_rs[fit_rs$id %in% ids_show, ], aes(x = time, y = mean)) +
  facet_wrap(~id) +
  geom_line(color = "steelblue") +
  geom_ribbon(aes(ymin = `0.025quant`, ymax = `0.975quant`),
    alpha = 0.2, fill = "steelblue") +
  geom_point(data = simDat[simDat$id %in% ids_show, ],
    aes(x = time, y = Y), size = 0.8) +
  labs(x = "Time (years)", y = "Marker",
    title = "Random intercept and slope model") +
  theme_minimal()
```

Random intercept and slope model



Priors for the model parameters

Two prior families.

- Fixed effects: Gaussian, mean 0, precision 0.01. Weakly informative. Argument `priorFixed`.
- Random-effects var-cov: Inverse Wishart. Argument `priorRandom`.

Check what this model used:

```
M_rs$priors_used$priorFixed
```

```
## $mean
## [1] 0
##
## $prec
## [1] 0.01
```

```
M_rs$priors_used$priorRandom
```

```
## $r
## [1] 3
##
## $R
## [1] 1
```

Informative data: the prior barely moves the posterior. Weak data: a strong prior can take over.

If the prior and the posterior land on top of each other for a parameter, what is the data telling us?

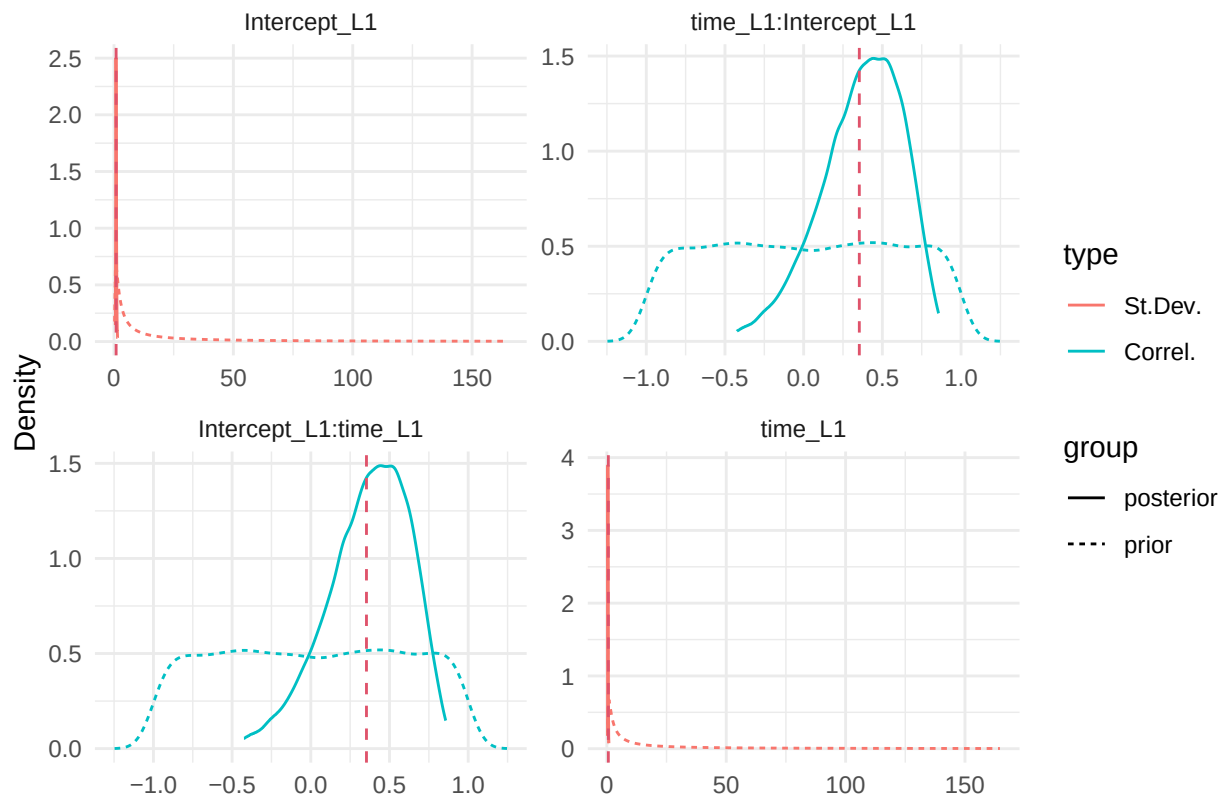
Compare the default prior to a strong one. Red lines are the truth.

```
M_inf <- joint(formLong = Y ~ time + (1 + time | id), dataLong = simDat,
              id = "id", timeVar = "time",
              control = list(int.strategy = "eb", priorRandom = list(r = 100)))

truth <- data.frame(
  Effect = unique(plot(M_rs, sdcor = TRUE)$Covariances$L1$data$Effect),
  TrueValues = c(cov2cor(Sigma) + diag(sqrt(diag(Sigma)) - 1)))
```

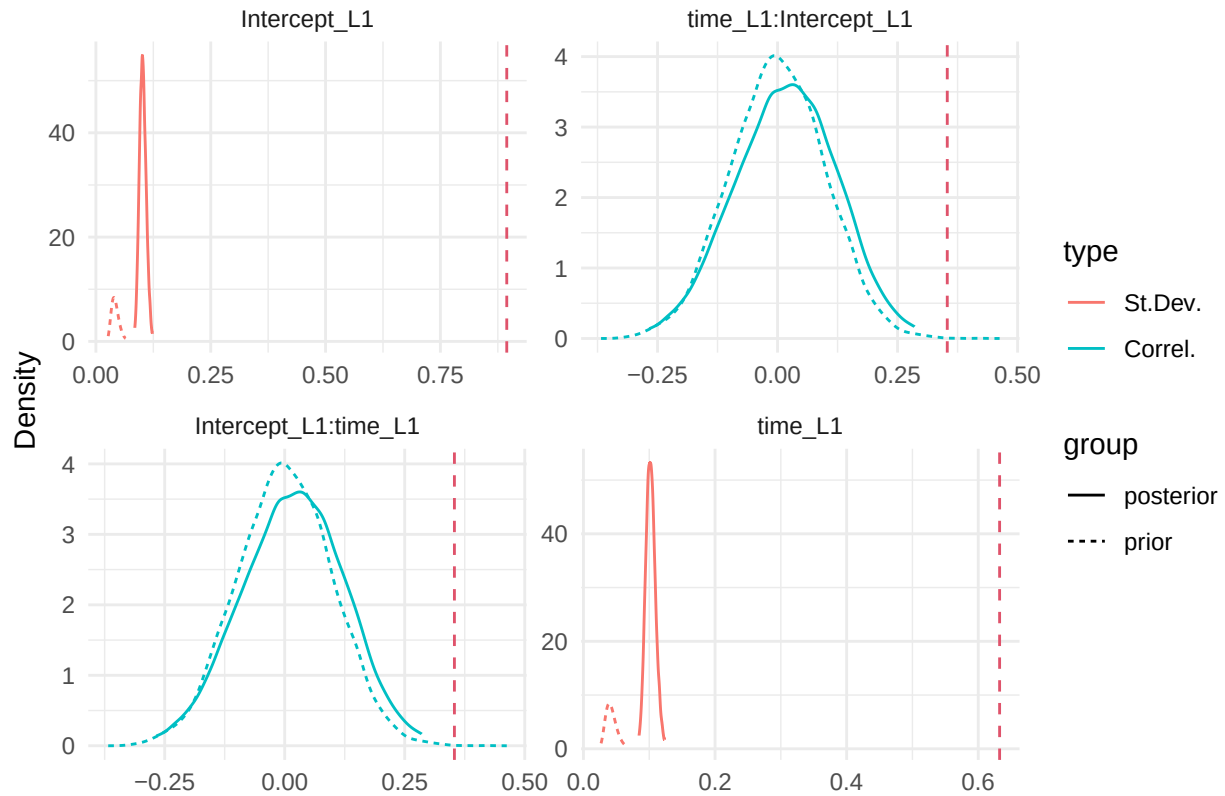
Default prior:

```
plot(M_rs, sdcor = TRUE, priors = TRUE)$Covariances$L1 +
  geom_vline(aes(xintercept = TrueValues), data = truth, colour = 2, linetype = 2)
```



Informative prior (r = 100), posterior pulled toward zero:

```
plot(M_inf, sdcor = TRUE, priors = TRUE)$Covariances$L1 +
  geom_vline(aes(xintercept = TrueValues), data = truth, colour = 2, linetype = 2)
```



Strong priors on random effects are sensitive, so check sensitivity before trusting them.

Prior sensitivity analysis can help detect identifiability issues, as well as how much the posterior is influenced by the prior.

Nonlinear trajectories

Any function of time can be considered to capture non-linear trajectories.

```
set.seed(3)
n <- 150
dat <- expand.grid(id = 1:n, time = 0:8)
dat <- dat[order(dat$id, dat$time), ]

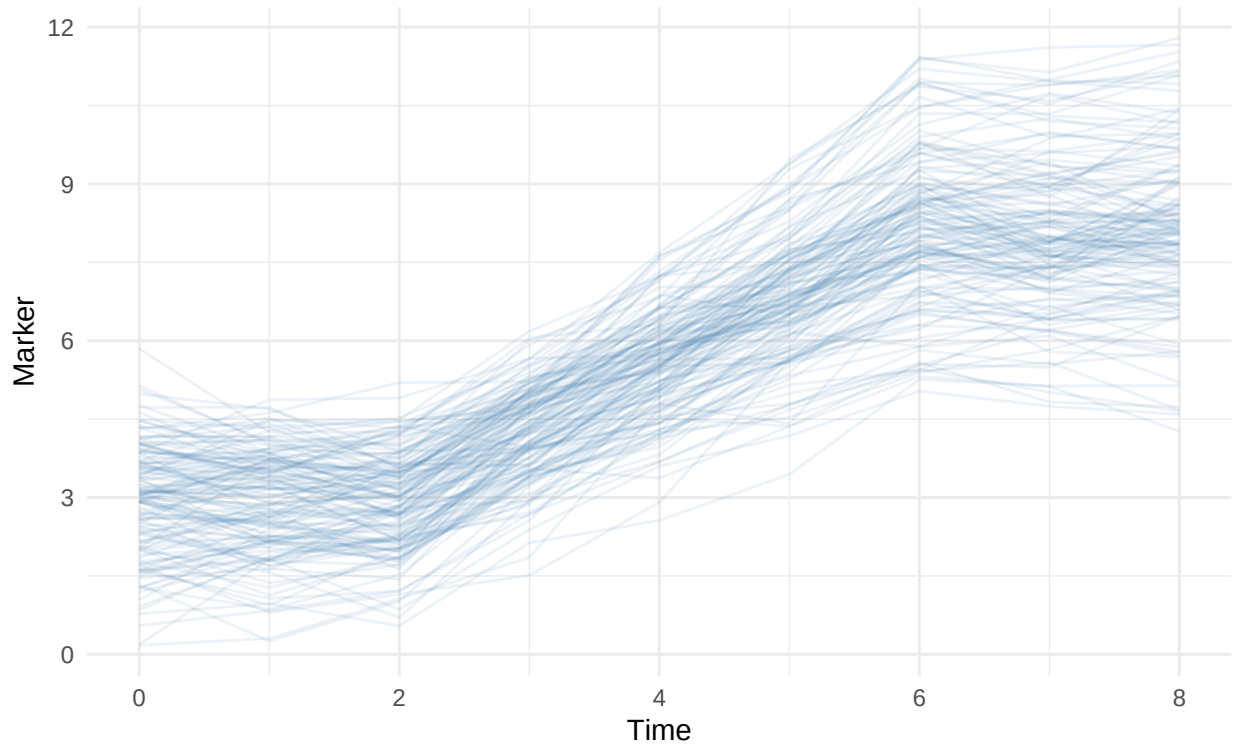
# shared timing: flat, a rise between t = 2 and t = 6, then flat again
ramp <- pmin(pmax((dat$time - 2) / 4, 0), 1)

lev <- rnorm(n, 3, 0.8)      # patient baseline
amp <- rnorm(n, 5, 1.0)     # patient rise height (subject-specific shape)
drift <- rnorm(n, 0, 0.15)  # patient late deviation

dat$Y <- rnorm(nrow(dat),
  lev[dat$id] + amp[dat$id] * ramp + drift[dat$id] * (dat$time - 4), 0.4)

ggplot(dat, aes(time, Y, group = id)) +
  geom_line(alpha = 0.10, color = "steelblue") +
```

```
labs(x = "Time", y = "Marker") +
theme_minimal()
```



We select 4 patients to check the fit:

```
sel <- order(amp)[c(1, 50, 100, 150)]
```

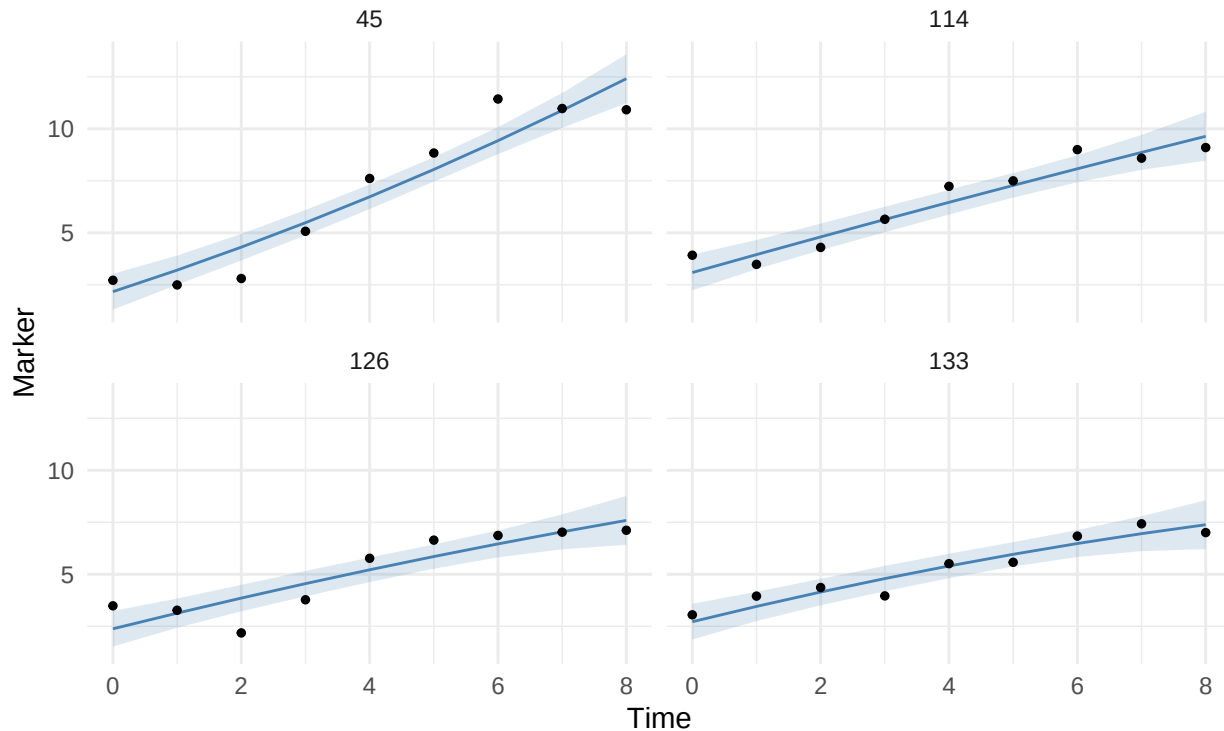
Let's plot some patient observed points against the model's fitted trajectory (one panel per patient).

```
show_fit <- function(model, data, ids) {
  fit <- cbind(data, model$summary.fitted.values)
  ggplot(fit[fit$id %in% ids, ], aes(time, mean)) +
    geom_ribbon(aes(ymin = `0.025quant`, ymax = `0.975quant`),
              fill = "steelblue", alpha = 0.18) +
    geom_line(color = "steelblue") +
    geom_point(aes(y = Y), size = 1) +
    facet_wrap(~id) +
    labs(x = "Time", y = "Marker") +
    theme_minimal()
}
```

```
f1 <- function(time) time^2
```

Let's try adding a quadratic term:

```
fit_quad <- joint(
  formLong = Y ~ time + f1(time) + (1 + time + f1(time) | id),
  dataLong = dat, id = "id", timeVar = "time",
  family = "gaussian"
)
show_fit(fit_quad, dat, sel)
```



What about spline functions?

Instead of one curve over the whole window, we let the trend be flexible only where it changes. Natural cubic splines with knots placed at the two change points, $t = 2$ and $t = 6$, can do this. They enter the formula through the same functions of time structure, in the fixed and the random part, so each patient again gets their own curve.

```
library(splines)

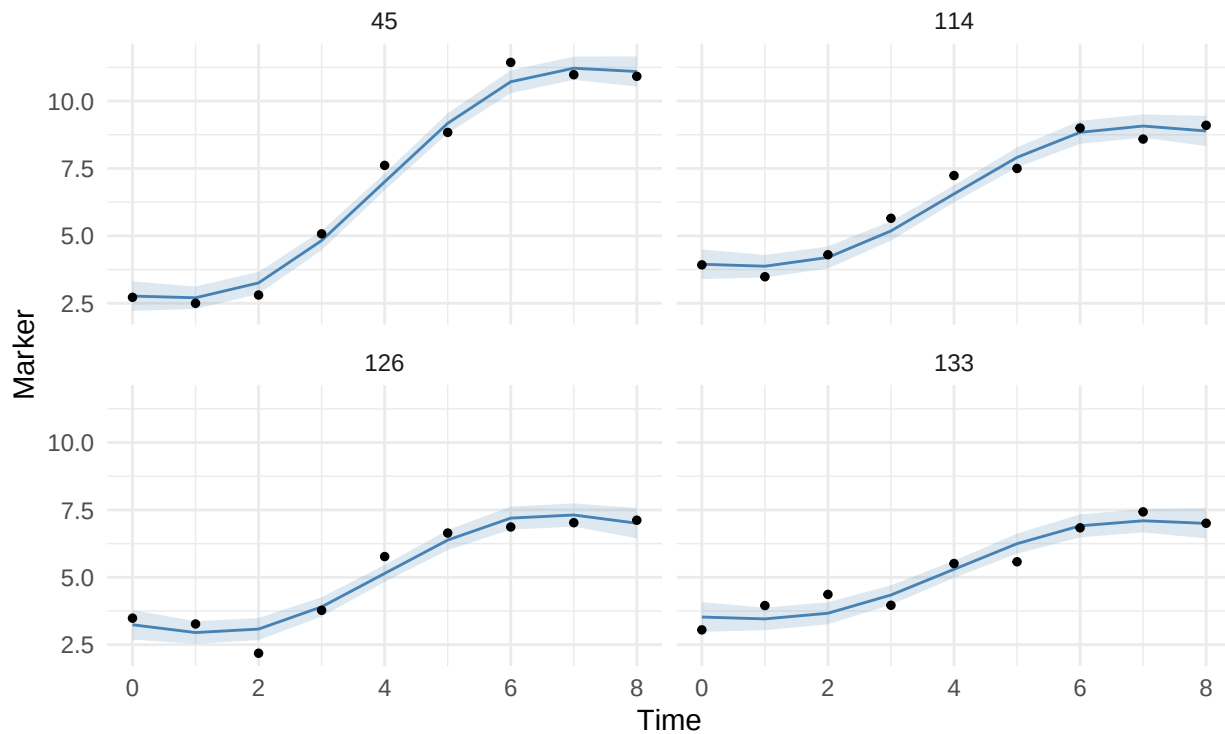
# knots at the two change points -> 3 basis columns
NS <- ns(dat$time, knots = c(2, 6))
f1 <- function(time) predict(NS, time)[, 1]
f2 <- function(time) predict(NS, time)[, 2]
f3 <- function(time) predict(NS, time)[, 3]

fit_spl <- joint(
  formLong = Y ~ f1(time) + f2(time) + f3(time) +
    (1 + f1(time) + f2(time) + f3(time) | id),
  dataLong = dat, id = "id", timeVar = "time",
  family = "gaussian"
)
summary(fit_spl, sdcor = TRUE)
```

```
## Longitudinal outcome (gaussian)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept      2.9661 0.0861      2.7971  2.9661    3.1351
## f1time          5.7851 0.1251      5.5393  5.7851    6.0308
## f2time          4.8011 0.1699      4.4674  4.8011    5.1346
## f3time          5.4327 0.1317      5.1741  5.4327    5.6912
## Res. err. (sd)  0.4205 0.0101      0.4013  0.4203    0.4408
```

```
##
## Random effects standard deviation / correlation (L1)
##          mean      sd 0.025quant 0.5quant 0.975quant
## Intercept    0.9834 0.0670    0.8657    0.9812    1.1203
## f1time        1.3922 0.0981    1.2017    1.3885    1.5973
## f2time        1.8036 0.1389    1.5366    1.8058    2.0749
## f3time        1.5444 0.0966    1.3606    1.5423    1.7422
## Intercept:f1time -0.3092 0.0923   -0.4729   -0.3138   -0.1088
## Intercept:f2time -0.5240 0.0752   -0.6560   -0.5307   -0.3676
## Intercept:f3time -0.3974 0.0834   -0.5445   -0.4034   -0.2184
## f1time:f2time    0.8508 0.0467    0.7388    0.8580    0.9186
## f1time:f3time    0.9453 0.0176    0.9041    0.9485    0.9694
## f2time:f3time    0.9232 0.0251    0.8672    0.9269    0.9593
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                               -1346.896                               -1338.546
##
## Deviance Information Criterion: 1890.582
## Widely applicable Bayesian information criterion: 1898.661
## Computation time: 1.72 seconds
```

```
show_fit(fit_spl, dat, sel) # flat, rise, flat: all three phases recovered
```



Model comparison based on WAIC:

```
joint.compare(fit_quad, fit_spl)
```

```
##      model1 model2 dWAIC   se    z p better sig
## 1 fit_quad fit_spl 1740.1 54.4 32.01 0 fit_spl **
```

Generalized longitudinal mixed models

Same fixed and random effects, different **family** and **link function**.

$$Y_{ij} \mid b_i \sim \text{distribution suited to the marker}, \quad g\{E(Y_{ij} \mid b_i)\} = \eta_i(t_{ij}).$$

Poisson mixed model for counts

For patient i at visit j ,

$$Y_{ij} \mid b_i \sim \text{Poisson}(\lambda_{ij}), \quad \log(\lambda_{ij}) = \eta_i(t_{ij}).$$

The coefficients are interpreted on the count scale through the exponential. A value of -0.20 means the expected count is multiplied by $\exp(-0.20)$, i.e., an $\sim 18\%$ drop.

Let's simulate some count data:

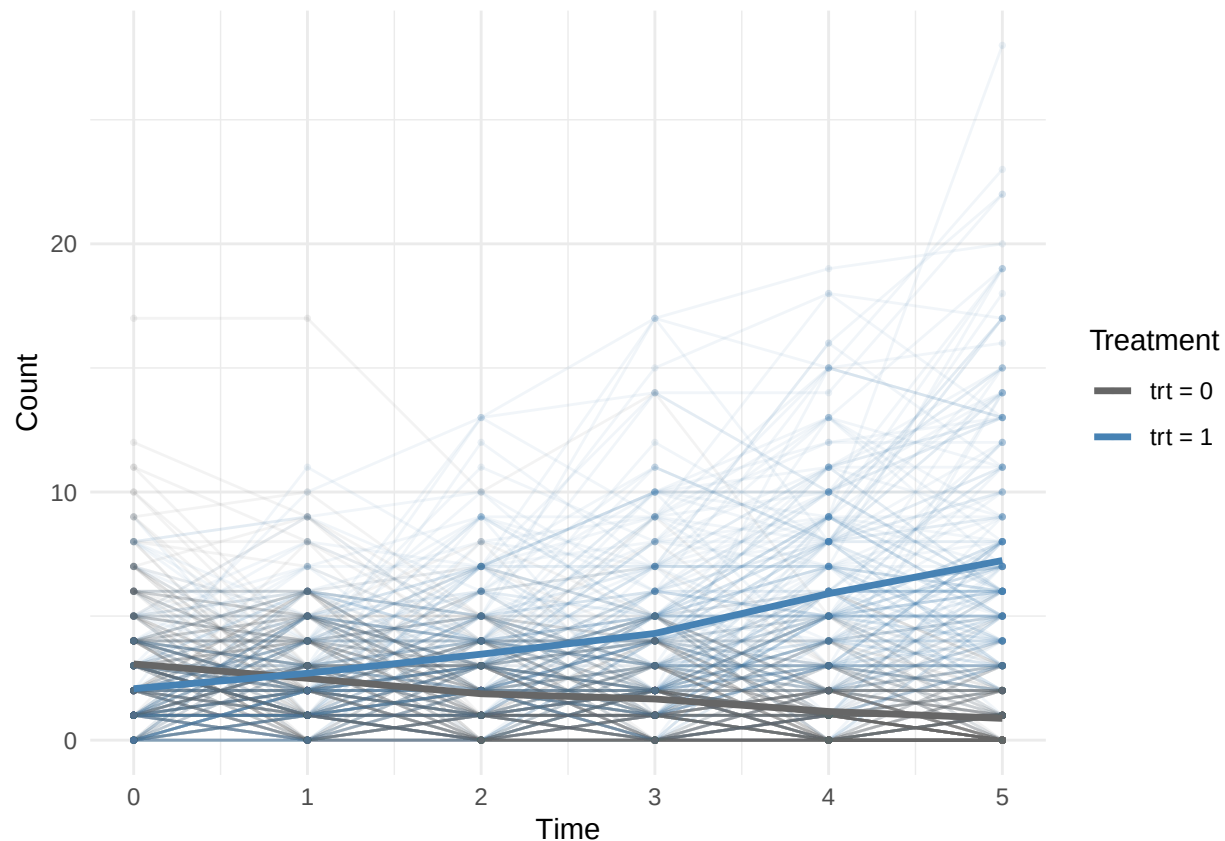
```
set.seed(1)
n_p <- 500
visit_times <- 0:5

datPois <- expand.grid(id = 1:n_p, time = visit_times)
datPois <- datPois[order(datPois$id, datPois$time), ]
datPois$trt <- rep(rbinom(n_p, 1, 0.5), each = length(visit_times))
datPois$Treatment <- factor(datPois$trt, levels = c(0, 1),
                             labels = c("trt = 0", "trt = 1"))

b_pois <- rep(rnorm(n_p, 0, 0.5), each = length(visit_times))
eta_pois <- 1 + b_pois - 0.25 * datPois$time - 0.35 * datPois$trt + 0.5 * datPois$time * datPois$trt
datPois$Y <- rpois(nrow(datPois), exp(eta_pois))

pois_mean <- aggregate(Y ~ time + Treatment, datPois, mean)

ggplot(datPois, aes(x = time, y = Y, group = id, color = Treatment)) +
  geom_line(alpha = 0.08) +
  geom_point(alpha = 0.10, size = 0.6) +
  geom_line(data = pois_mean, aes(group = Treatment), linewidth = 1.2) +
  labs(x = "Time", y = "Count") +
  scale_color_manual(values = c("gray40", "steelblue")) +
  theme_minimal()
```



The model fit is similar, we just need to define the appropriate family:

```
M_pois <- joint(
  formLong = Y ~ time * trt + (1 | id),
  dataLong = datPois,
  id = "id",
  timeVar = "time",
  family = "poisson"
)
summary(M_pois, sdcor = TRUE)
```

```
## Longitudinal outcome (poisson)
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept  1.0012 0.0430    0.9165   1.0013   1.0852
## time       -0.2424 0.0113   -0.2645  -0.2424  -0.2203
## trt        -0.4056 0.0625   -0.5280  -0.4056  -0.2829
## time:trt    0.4940 0.0139    0.4668   0.4940   0.5211
##
## Random effects standard deviation / correlation (L1)
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept  0.5249 0.0213    0.4844   0.5244   0.5682
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                               -5725.636                               -5726.026
##
## Deviance Information Criterion: 10970.24
## Widely applicable Bayesian information criterion: 10992.79
```

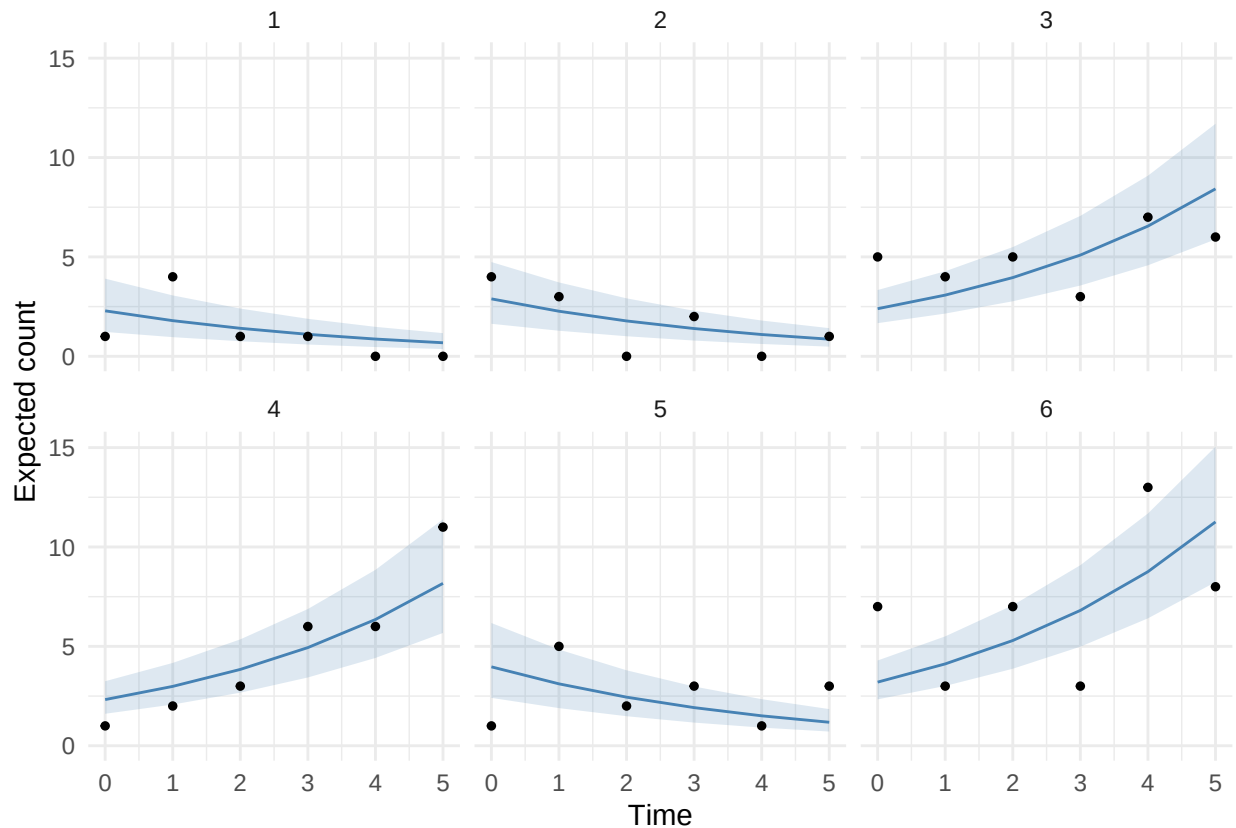
Computation time: 0.65 seconds

- `trt`: treatment log-count ratio at time 0. Exponentiate for the count ratio.
- `time:trt`: difference in time slopes between arms.
- Random-intercept SD: between-patient variation on the log-count scale.

For a Gaussian marker the link is the identity. For counts and yes/no markers, `summary.fitted.values` already applies the inverse link, so the fitted values and intervals are on the count or probability scale.

```
pois_ids <- unique(datPois$id)[1:6]
fit_pois <- cbind(datPois, M_pois$summary.fitted.values)

ggplot(fit_pois[fit_pois$id %in% pois_ids, ], aes(x = time, y = mean)) +
  geom_ribbon(aes(ymin = `0.025quant`, ymax = `0.975quant`),
            fill = "steelblue", alpha = 0.18) +
  geom_line(color = "steelblue") +
  geom_point(aes(y = Y), size = 1) +
  facet_wrap(~id) +
  labs(x = "Time", y = "Expected count") +
  theme_minimal()
```



The curve is the posterior mean of λ_{ij} , with a 95% credible interval for the underlying mean count.

Binomial mixed model for binary outcomes

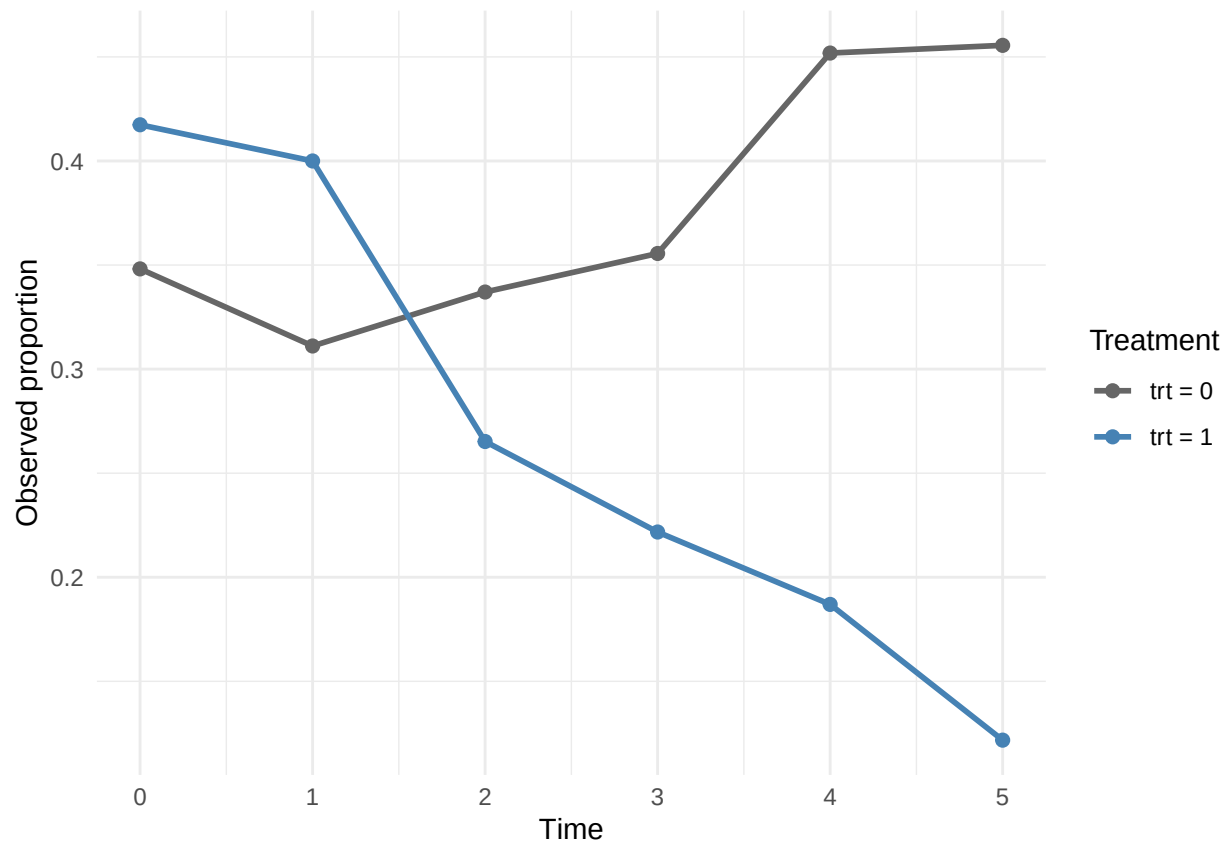
Binary outcomes (0/1), `family = "binomial"` and a logit link.

$$Y_{ij} \mid b_i \sim \text{Bernoulli}(p_{ij}), \quad \text{logit}(p_{ij}) = \eta_i(t_{ij}).$$

```
datBinom <- datPois[, c("id", "time", "trt", "Treatment")]
eta_binom <- -1 + b_pois + 0.15 * datBinom$time + 0.70 * datBinom$trt - 0.50 * datBinom$time * datBinom$trt
datBinom$Y <- rbinom(nrow(datBinom), size = 1, prob = plogis(eta_binom))
```

```
tox_prop <- aggregate(Y ~ time + Treatment, datBinom, mean)
```

```
ggplot(tox_prop, aes(x = time, y = Y, color = Treatment)) +
  geom_point(size = 2) +
  geom_line(linewidth = 1) +
  labs(x = "Time", y = "Observed proportion") +
  scale_color_manual(values = c("gray40", "steelblue")) +
  theme_minimal()
```



```
M_binom <- joint(
  formLong = Y ~ time * trt + (1 | id),
  dataLong = datBinom,
  id = "id",
  timeVar = "time",
  family = "binomial"
)
summary(M_binom, sdcor = TRUE)
```

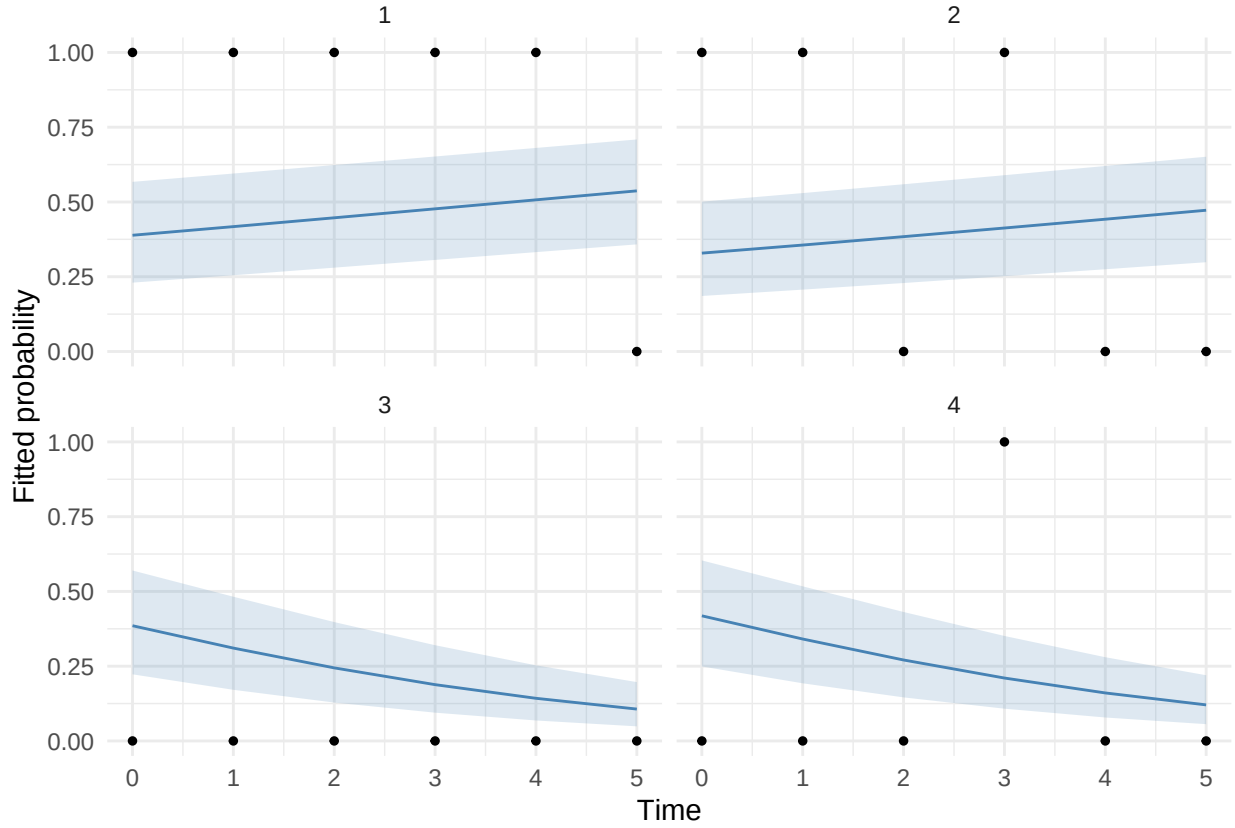
```
## Longitudinal outcome (binomial)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept -0.8375 0.0979   -1.0302   -0.8373   -0.6461
## time       0.1240 0.0305    0.0642    0.1240    0.1837
```

```
## trt          0.5787 0.1427      0.2993  0.5786      0.8590
## time:trt    -0.4637 0.0493     -0.5605 -0.4637     -0.3670
##
## Random effects standard deviation / correlation (L1)
##           mean  sd 0.025quant 0.5quant 0.975quant
## Intercept 0.3598 0.1      0.1583  0.3679      0.5394
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                               -1852.154                               -1852.659
##
## Deviance Information Criterion: 3646.869
## Widely applicable Bayesian information criterion: 3646.526
## Computation time: 0.7 seconds
```

- trt: treatment log-odds ratio at time 0. The simulated odds ratio is $\exp(0.70) \approx 2.01$.
- time:trt: change in the treatment log-odds ratio per unit time.

```
binom_ids <- unique(datBinom$id)[1:4]
fit_binom <- cbind(datBinom, M_binom$summary.fitted.values)

ggplot(fit_binom[fit_binom$id %in% binom_ids, ], aes(x = time, y = mean)) +
  geom_ribbon(aes(ymin = `0.025quant`, ymax = `0.975quant`),
            fill = "steelblue", alpha = 0.18) +
  geom_line(color = "steelblue") +
  geom_point(aes(y = Y), size = 1) +
  facet_wrap(~id) +
  labs(x = "Time", y = "Fitted probability") +
  theme_minimal()
```



The points are 0 and 1. The line and 95% credible band describe each patient's fitted probability of $Y = 1$.

Ordinal mixed model

Many outcomes are categorized as ordered grades.

- a grade is ordered, Low below Moderate below High
- code them 1, 2, 3 and a straight line assumes the step Low to Moderate equals Moderate to High
- it can predict 2.7, a grade that does not exist

What the proportional-odds model changes:

- it works on cumulative probabilities, $P(Y \leq k)$
- a set of ordered cut-points marks the thresholds between grades
- one slope per covariate shifts every threshold together, the proportional-odds idea
- order is respected, equal spacing is not assumed

$$\text{logit}\{P(Y_{ij} \leq k \mid b_i)\} = \alpha_k - \eta_i(t_{ij}).$$

The cut-points α_k set the boundaries. The linear predictor slides probability mass toward lower or higher grades.

Written out for K ordered grades y_k ($k = 1, \dots, K$), the proportional odds model is

$$F(k) = P(Y \leq k) = \frac{\exp(\gamma_k)}{1 + \exp(\gamma_k)}, \quad \gamma_k = \alpha_k - \eta_i(t),$$

where the α_k form an increasing sequence of $K - 1$ cut-points,

$$\alpha_0 = -\infty < \alpha_1 < \alpha_2 < \dots < \alpha_{K-1} < \alpha_K = +\infty,$$

and the linear predictor carries the covariates and the random effects, for example considering a random intercept:

$$\eta_i(t) = b_{i0} + \beta_1 t + \beta_2 trt_i + \beta_3 trt_i t.$$

We simulate from this model with four cut-points, giving five ordered grades that we read as grades 1 (mildest) to 5 (most severe). Here treatment and time both raise the linear predictor, so both push probability mass toward the worse grades.

```
set.seed(4)

# draw an ordinal outcome from a proportional odds model
rpom <- function(alpha, eta) { # alpha: cut-points, eta: linear predictor
  Flog <- function(x) 1 / (1 + exp(-x)) # logistic CDF -> logit link
  ns <- length(eta)
  nc <- length(alpha) + 1
  y <- numeric(ns)
  for (k in 1:ns) {
    p <- diff(c(0, Flog(alpha - eta[k])), 1))
    y[k] <- sample(1:nc, 1, prob = p)
  }
  y
}

datOrd <- datPois[, c("id", "time", "trt", "Treatment")]
linPred <- b_pois + 0.1 * datOrd$time + 1 * datOrd$trt
alpha <- quantile(linPred, c(0.2, 0.4, 0.6, 0.8))

datOrd$Grade <- rpom(alpha, linPred)
datOrd$trt <- factor(datOrd$trt)
```

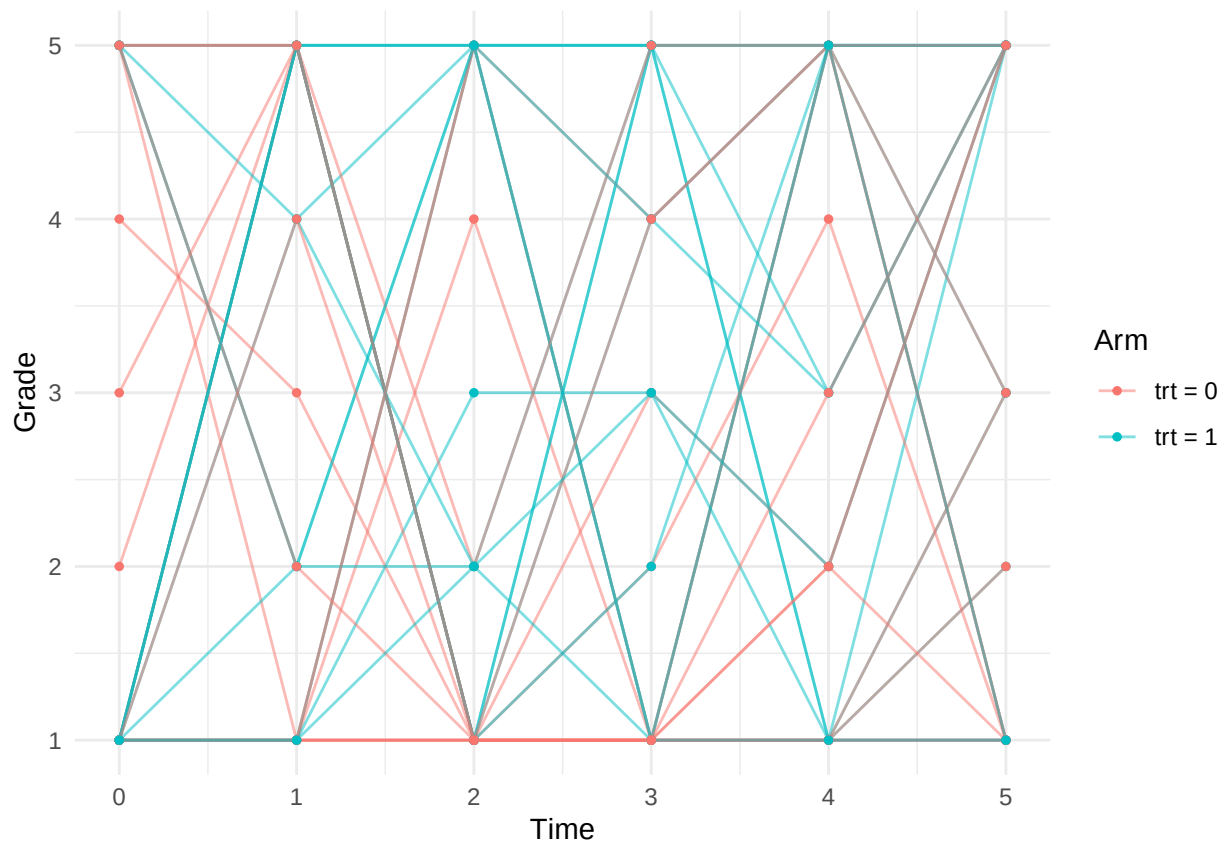
The outcome is an ordered grade, spread across the five levels:

```
table(datOrd$Grade)

##
##      1      2      3      4      5
## 1070   275   253   305  1097
```

Individual trajectories over follow-up (a random sample of patients):

```
set.seed(1)
show <- sample(unique(datOrd$id), 30)
ggplot(subset(datOrd, id %in% show),
  aes(time, Grade, group = id, colour = Treatment)) +
  geom_line(alpha = 0.5) + geom_point(size = 1) +
  scale_y_continuous(breaks = 1:5) +
  labs(x = "Time", y = "Grade", colour = "Arm") +
  theme_minimal()
```



We fit the model with `family = "pom"` and a logit link. Since `logit` is already the default for the `pom` family, writing `link = "logit"` here just makes the choice explicit.

```
M_pom <- joint(formLong = Grade ~ -1 + time * trt + (1 | id),
               dataLong = datOrd, id = "id", timeVar = "time",
               family = "pom", link = "logit")
summary(M_pom, sdcor = TRUE)
```

```
## Longitudinal outcome (pom)
##               mean      sd 0.025quant 0.5quant 0.975quant
## theta1 for POM  0.0871 0.0880   -0.0861   0.0871    0.2602
## theta2 for POM -0.8482 0.0582   -0.9631  -0.8481   -0.7338
## theta3 for POM -0.9688 0.0608   -1.0893  -0.9686   -0.8497
## theta4 for POM -0.7567 0.0552   -0.8657  -0.7566   -0.6485
## time           0.1264 0.0268    0.0736   0.1263    0.1795
## trt1           0.9873 0.1280    0.7366   0.9869    1.2400
## time:trt1      -0.0166 0.0397   -0.0945  -0.0166    0.0614
##
## Random effects standard deviation / correlation (L1)
##               mean      sd 0.025quant 0.5quant 0.975quant
## Intercept 0.4672 0.0592    0.3567   0.4658    0.5887
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##               -4109.573                    -4106.466
##
## Deviance Information Criterion: 8115.787
## Widely applicable Bayesian information criterion: 8119.886
```

```
## Computation time: 1.02 seconds
```

Here `pom` is INLA's proportional-odds family. The cut-points take the place of a single intercept, which is why the formula drops the global intercept.

INLA stores the cut-points through an internal parameterization, so we convert the sampled `theta` values back to the α_k and compare them with the values used to simulate:

```
theta <- inla.hyperpar.sample(1e5, M_pom, intern = TRUE)
nms    <- paste(paste0("theta", 1:length(alpha)), "for POM")

sim.alpha <- matrix(NA, nrow(theta), length(alpha))
for (k in 1:length(alpha)) {
  if (k == 1) {
    sim.alpha[, k] <- theta[, nms[1]]
  } else {
    sim.alpha[, k] <- sim.alpha[, k - 1] + exp(theta[, nms[k]])
  }
}
colnames(sim.alpha) <- paste0("alpha", 1:length(alpha))

m1 <- colMeans(sim.alpha)
m2 <- colMeans(sim.alpha^2)
print(cbind(true = alpha, estimate = m1, stdev = sqrt(m2 - m1^2)))

##           true      estimate      stdev
## 20% 0.0398385 0.08669412 0.08748090
## 40% 0.4811551 0.51490646 0.08812016
## 60% 0.8760647 0.89433671 0.08946312
## 80% 1.3205586 1.36359532 0.09178616
```

The recovered cut-points are close to the truth: these are the boundaries that separate the five grades on the latent scale, while the fixed effects measure how covariates move a patient toward a higher (worse) grade.

For treatment, the odds ratio at baseline is $\exp(\beta_2)$, so `trt = 1` carries about

```
round(exp(M_pom$summary.fixed[2, 1]), 2)

## [1] 2.68
```

times the odds of being in a worse grade than the reference arm. Because of the proportional-odds structure, “a worse grade” means the same shift applied at every threshold at once: the odds of grade ≥ 2 versus grade 1, of grade ≥ 3 versus grades ≤ 2 , and so on up to grade 5 versus grades 1 to 4. A single odds ratio therefore summarizes the treatment's tendency to push patients toward higher grade across all grade boundaries.

Multiple longitudinal markers

Consider tumor size (continuous), lesion count (Poisson) and toxicity (binary), measured in the same patients.

Each marker has its own regression model and link. `corLong = TRUE` allows the marker-specific random effects to correlate:

$$\begin{aligned} E[\text{Size}_i(t)] &= \beta_{10} + b_{1i} + \beta_{11}t + \beta_{12}trt_i + \beta_{13}trt_it \\ \log E[\text{Count}_i(t)] &= \beta_{20} + b_{2i} + \beta_{21}t + \beta_{22}trt_i + \beta_{23}trt_it \\ \logit P[\text{Tox}_i(t) = 1] &= \beta_{30} + b_{3i} + \beta_{31}t + \beta_{32}trt_i + \beta_{33}trt_it \end{aligned}$$

The random intercepts follow $(b_{1i}, b_{2i}, b_{3i}) \sim \mathcal{N}(0, \Sigma)$. The off-diagonal terms link the markers. Tumor size also includes Gaussian residual error.

```
set.seed(1)

n <- 200 # patients
ni <- 6 # visits per patient (times 0..5)

# true cross-marker random-intercept covariance (size, count, toxicity)
Sigma <- matrix(c(0.50, 0.25, 0.15,
                  0.25, 0.40, 0.20,
                  0.15, 0.20, 0.35), nrow = 3)

b <- mvtnorm::rmvnorm(n, mean = c(0, 0, 0), sigma = Sigma)

dat <- expand.grid(time = 0:5, id = 1:n)
dat$trt <- rep(rbinom(n, 1, 0.5), each = ni)
dat$Treatment <- factor(dat$trt, levels = c(0, 1),
                        labels = c("trt = 0", "trt = 1"))

b1 <- rep(b[, 1], each = ni) # tumor-size intercepts
b2 <- rep(b[, 2], each = ni) # lesion-count intercepts
b3 <- rep(b[, 3], each = ni) # severe-toxicity intercepts

# tumor size
dat$Size <- rnorm(nrow(dat), 5 + b1 - 0.30 * dat$time - 0.50 * dat$trt + 0.5 * dat$time * dat$trt, 0.5)

# lesion count
dat$Count <- rpois(nrow(dat), exp(1.2 + b2 + 0.10 * dat$time + 0.80 * dat$trt + 0.03 * dat$time * dat$trt))

# severe toxicity
eta3 <- -1.0 + b3 + 0.15 * dat$time + 0.50 * dat$trt + 0.2 * dat$time * dat$trt
dat$Tox <- rbinom(nrow(dat), size = 1, prob = 1 / (1 + exp(-eta3)))
```

The three markers evolve together, one row per visit:

```
head(dat, 10)
```

##	time	id	trt	Treatment	Size	Count	Tox
## 1	0	1	0	trt = 0	4.176171	2	0
## 2	1	1	0	trt = 0	5.215741	4	0
## 3	2	1	0	trt = 0	3.884956	4	0
## 4	3	1	0	trt = 0	3.622947	4	1
## 5	4	1	0	trt = 0	2.768227	7	0
## 6	5	1	0	trt = 0	2.357890	1	0
## 7	0	2	1	trt = 1	4.796964	11	0
## 8	1	2	1	trt = 1	5.547558	12	0
## 9	2	2	1	trt = 1	6.639005	21	1
## 10	3	2	1	trt = 1	7.035440	12	1

Plotting them side by side:

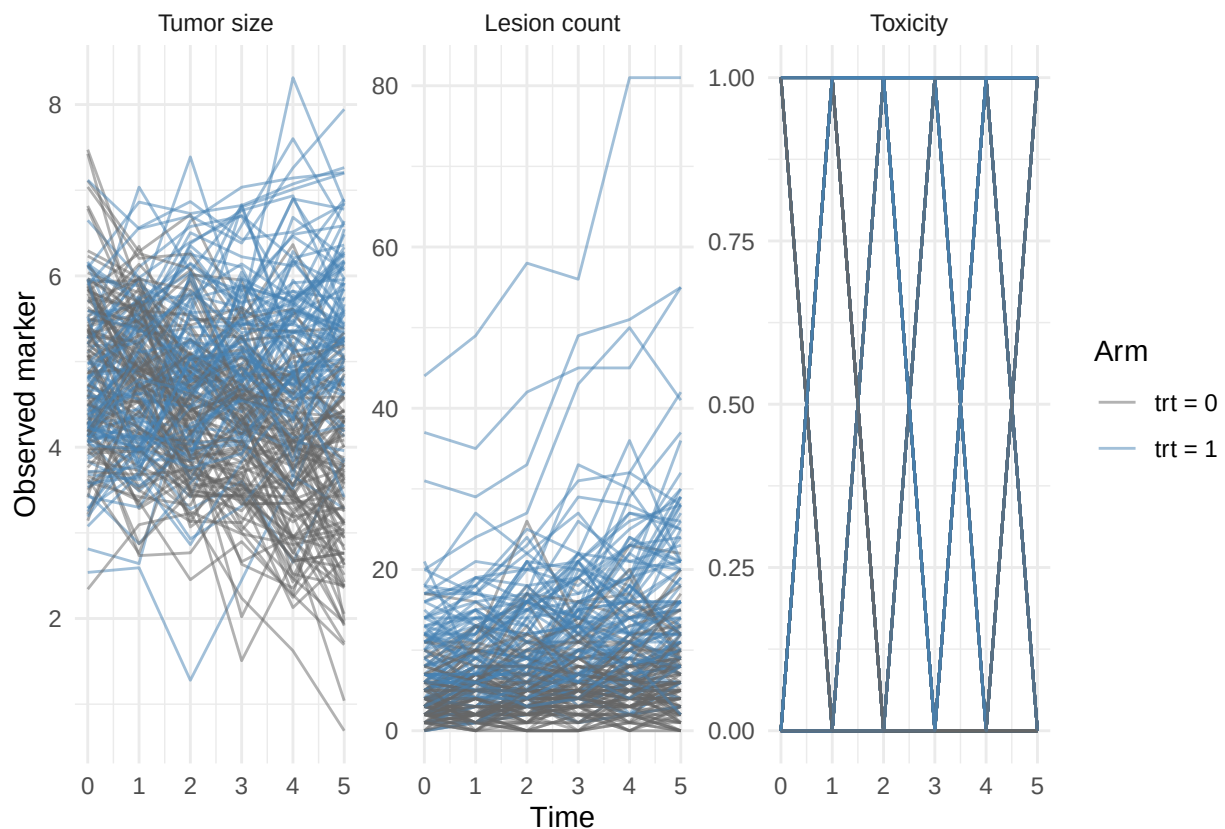
```
datPlot <- rbind(
  data.frame(id = dat$id, time = dat$time, Treatment = dat$Treatment,
             Marker = "Tumor size", Value = dat$Size),
  data.frame(id = dat$id, time = dat$time, Treatment = dat$Treatment,
```

```

      Marker = "Lesion count", Value = dat$Count),
    data.frame(id = dat$id, time = dat$time, Treatment = dat$Treatment,
      Marker = "Toxicity", Value = dat$Tox)
)
datPlot$Marker <- factor(datPlot$Marker, levels = c("Tumor size", "Lesion count", "Toxicity"))

ggplot(datPlot, aes(time, Value, group = id, colour = Treatment)) +
  geom_line(alpha = 0.5) +
  facet_wrap(~ Marker, scales = "free_y") +
  scale_colour_manual(values = c("gray40", "steelblue")) +
  labs(x = "Time", y = "Observed marker", colour = "Arm") +
  theme_minimal()

```



A list of formulas, one family per marker and `corLong = TRUE`:

```

M_multi <- joint(
  formLong = list(
    Size ~ time * trt + (1 | id),
    Count ~ time * trt + (1 | id),
    Tox ~ time * trt + (1 | id)
  ),
  dataLong = dat, id = "id", timeVar = "time",
  family = c("gaussian", "poisson", "binomial"),
  corLong = TRUE
)
summary(M_multi, sdcor = TRUE)

```

```
## Longitudinal outcome (L1, gaussian)
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1  5.0251 0.0812    4.8658  5.0251    5.1844
## time_L1      -0.3000 0.0121   -0.3238 -0.3000   -0.2762
## trt_L1       -0.4611 0.1178   -0.6921 -0.4611   -0.2300
## time:trt_L1   0.4912 0.0176    0.4567  0.4912    0.5257
## Res. err. (sd) 0.5193 0.0115    0.4971  0.5192    0.5422
##
## Longitudinal outcome (L2, poisson)
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L2  1.2472 0.0693    1.1108  1.2474    1.3828
## time_L2       0.0973 0.0101    0.0774  0.0973    0.1171
## trt_L2        0.8259 0.0966    0.6367  0.8258    1.0158
## time:trt_L2   0.0327 0.0122    0.0087  0.0327    0.0566
##
## Longitudinal outcome (L3, binomial)
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L3 -1.0009 0.1720   -1.3399 -1.0004   -0.6651
## time_L3      0.1131 0.0508    0.0136  0.1131    0.2127
## trt_L3       0.2596 0.2437   -0.2179  0.2594    0.7380
## time:trt_L3  0.2812 0.0747    0.1347  0.2811    0.4279
##
## Random effects standard deviation / correlation
##           mean      sd 0.025quant 0.5quant 0.975quant
## Intercept_L1  0.7404 0.0423    0.6640  0.7391    0.8283
## Intercept_L2  0.6131 0.0353    0.5456  0.6120    0.6840
## Intercept_L3  0.6690 0.0931    0.5036  0.6620    0.8702
## Intercept_L1:Intercept_L2 0.5556 0.0543    0.4428  0.5581    0.6564
## Intercept_L1:Intercept_L3 0.4392 0.1014    0.2243  0.4452    0.6111
## Intercept_L2:Intercept_L3 0.6034 0.0902    0.4032  0.6133    0.7523
##
## log marginal-likelihood (integration)    log marginal-likelihood (Gaussian)
##                               -5073.658                               -5068.792
##
## Deviance Information Criterion: 9320.293
## Widely applicable Bayesian information criterion: 9325.606
## Computation time: 2.36 seconds
```

The correlations describe how patient-specific deviations relate across markers, after accounting for time and treatment.

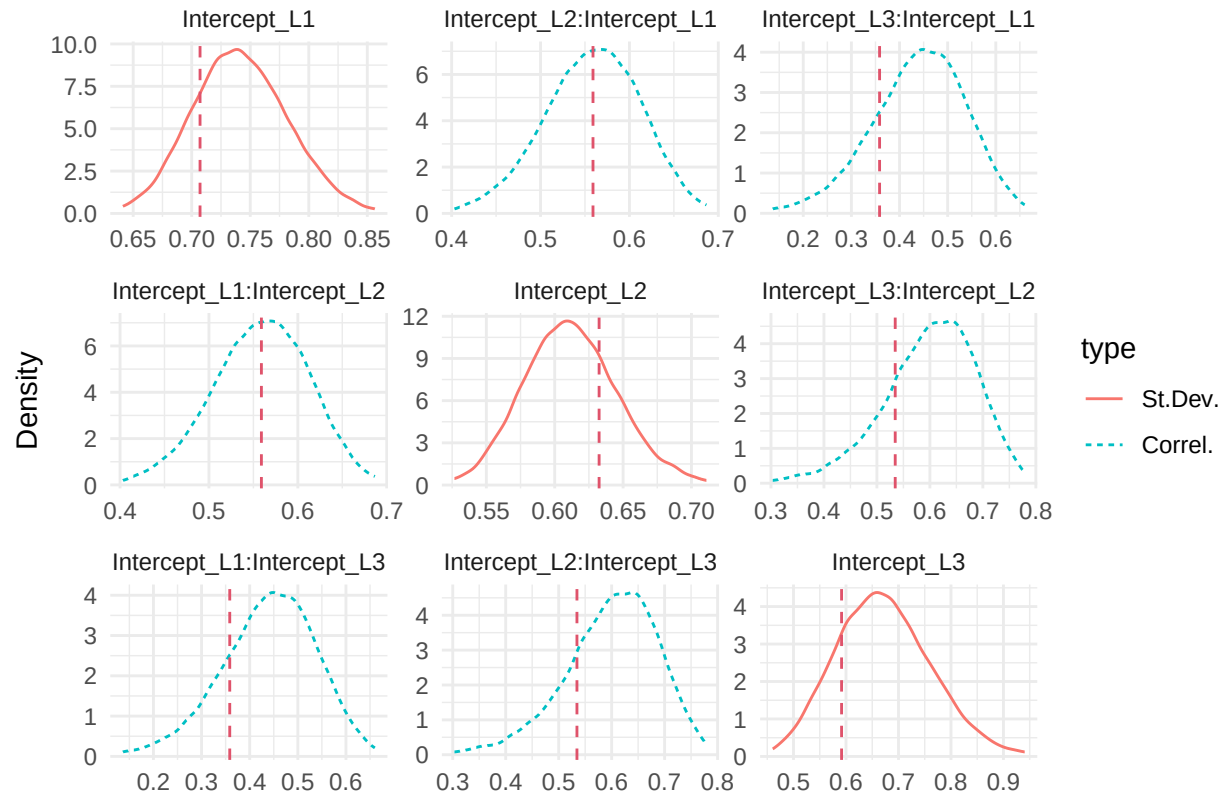
Recovering the correlation structure

Posterior standard deviations and correlations. Red dashed lines: simulated values.

```
p <- plot(M_multi, sdcor = TRUE)$Covariances$L1

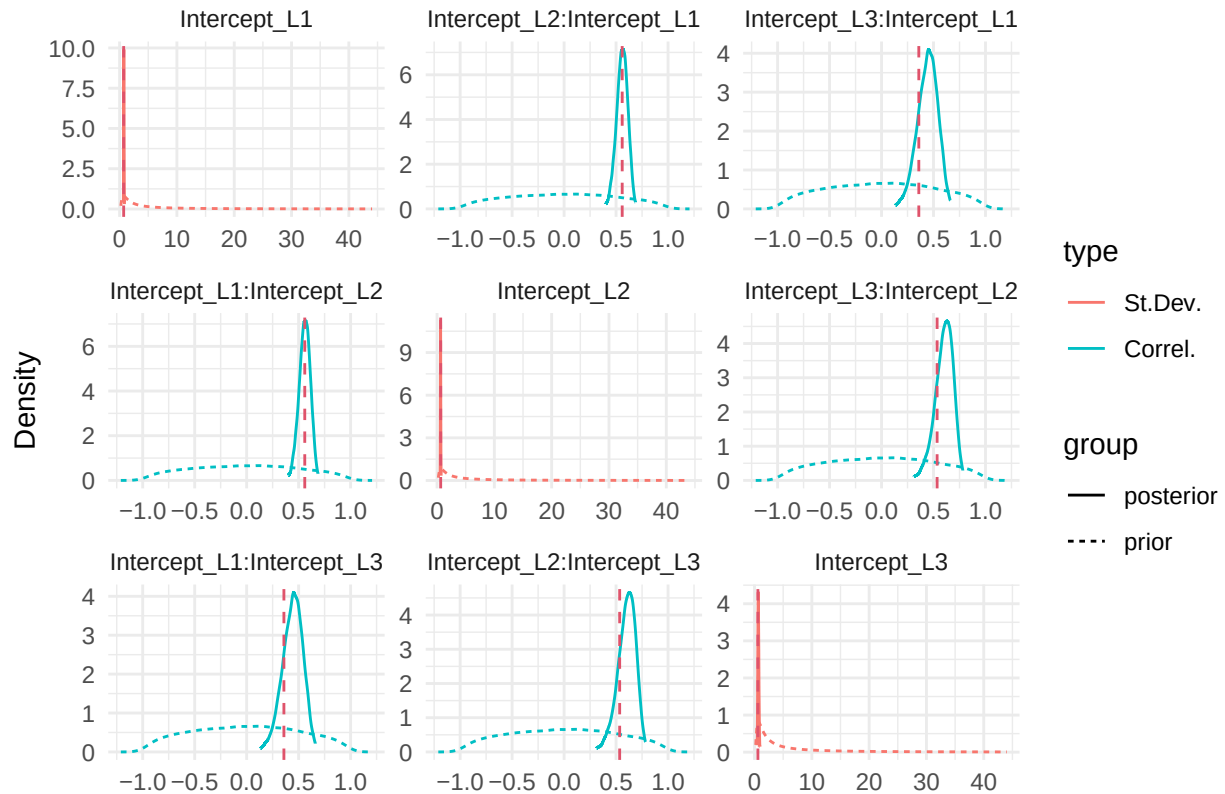
# true values on the sd/correlation scale: sds on the diagonal, correlations off it
trueSDcor <- cov2cor(Sigma) + diag(sqrt(diag(Sigma)) - 1)

p + geom_vline(
  data = data.frame(Effect = unique(p$data$Effect), TrueValues = c(trueSDcor)),
  aes(xintercept = TrueValues), colour = 2, linetype = 2)
```



Same with priors:

```
p <- plot(M_multi, sdcor = TRUE, priors = TRUE)$Covariances$L1
p + geom_vline(
  data = data.frame(Effect = unique(p$data$Effect), TrueValues = c(trueSDcor)),
  aes(xintercept = TrueValues), colour = 2, linetype = 2)
```



The toxicity random-effect variance is less precisely estimated in this example.

Interpretation

Read each coefficient on its own scale. The simulated effects are:

- Tumor size: effects in centimeters. Time slopes of -0.30 for `trt = 0` and +0.20 for `trt = 1`.
- Lesion count: baseline count ratio $\exp(0.80) \approx 2.23$ for `trt = 1` versus `trt = 0`.
- Toxicity: baseline odds ratio $\exp(0.50) \approx 1.65$ for `trt = 1` versus `trt = 0`.

Positive cross-marker correlations: patients above average on one marker tend also to be above average on the others. This allows information from one marker to inform another.

Predictions for a new patient

For a new patient, `predict()` uses the available measurements. Markers can have different observation times, with NA for unobserved values.

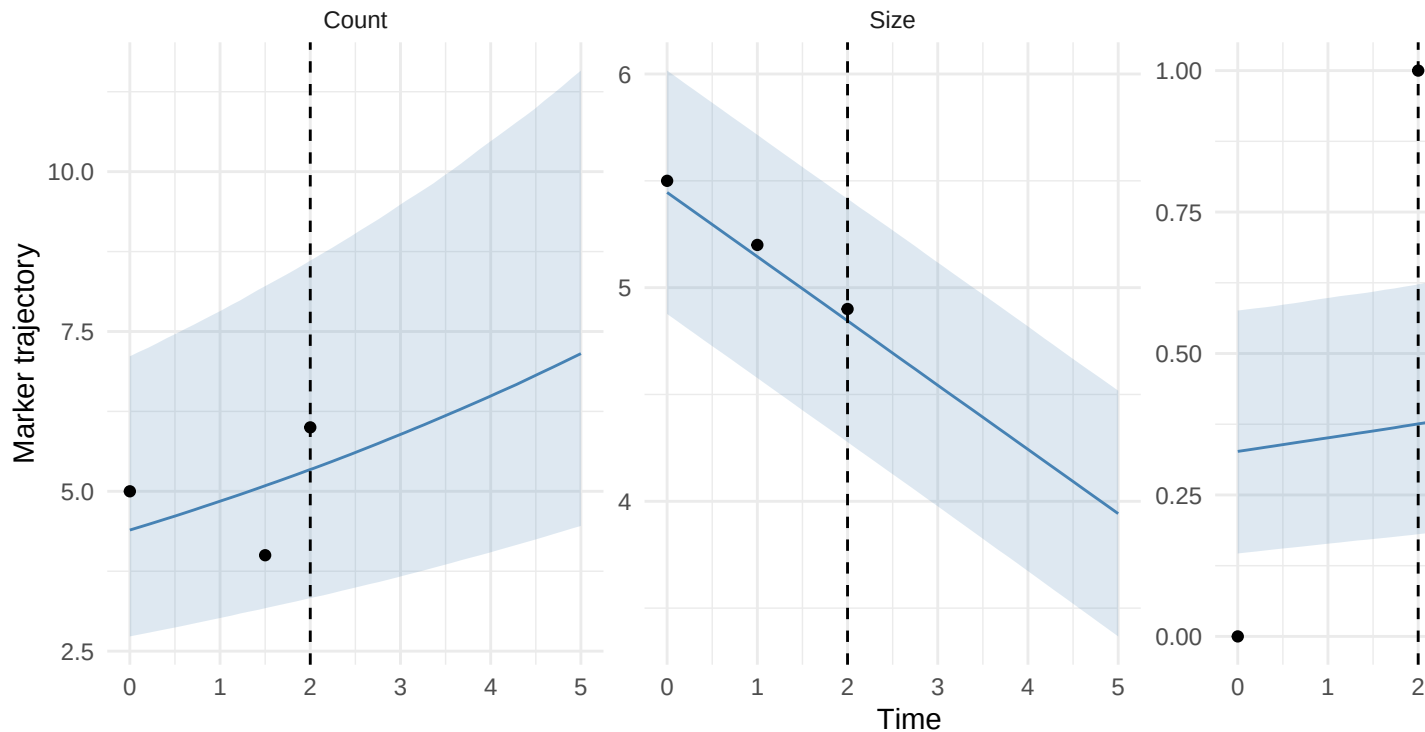
```
new_multi <- data.frame(
  id = 201, time = c(0, 1, 1.5, 2), trt = 0,
  Size = c(5.5, 5.2, NA, 4.9),
  Count = c(5, NA, 4, 6),
  Tox = c(0, NA, NA, 1)
)

set.seed(1)
pred_all <- predict(M_multi, newData = new_multi, horizon = 5,
  id = "id", inv.link = TRUE)
```

`inv.link = TRUE` returns the trajectories on the marker scale: tumor size, expected lesion count and toxicity probability.

```
new_multi_plot <- data.frame(
  time = rep(new_multi$time, 3),
  Outcome = rep(c("Size", "Count", "Tox"), each = nrow(new_multi)),
  value = unlist(new_multi[, c("Size", "Count", "Tox")])
)

ggplot(pred_all$PredL, aes(time, quant0.5)) +
  geom_ribbon(aes(ymin = quant0.025, ymax = quant0.975),
    fill = "steelblue", alpha = 0.18) +
  geom_line(color = "steelblue") +
  geom_point(data = new_multi_plot, aes(y = value), size = 1.5, na.rm = TRUE) +
  geom_vline(xintercept = 2, linetype = 2) +
  facet_wrap(~Outcome, scales = "free_y") +
  labs(x = "Time", y = "Marker trajectory") +
  theme_minimal()
```



The points are the new patient's measurements. The dashed line marks the last observed visit. The bands describe uncertainty in the underlying trajectories. For Gaussian longitudinal outcomes, `resErrLong = TRUE` adds residual error to the prediction distribution for a future measurement.

This correlation is the same machinery that powers the joint model.

- one patient, several markers, tied through shared or correlated random effects
- each marker can become a longitudinal submodel
- those same random effects can inform the risk of death through the association term

Longitudinal session summary

- Random intercepts and slopes capture patient heterogeneity around population-average trajectories.
- Flexible time effects extend mixed models to nonlinear trajectories.
- The same framework handles Gaussian, count, binary and ordinal longitudinal outcomes.
- Multiple markers can be modeled together through correlated random effects and used for individual prediction.

The next session combines longitudinal and survival processes in joint models.